ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

Τεχνικό Χρέος

Ποιότητα Λογισμικού

Απόστολος Αμπατζόγλου
Επίκουρος Καθηγητής
a.ampatzoglou@uom.edu.gr

"Shipping first time code is like going into debt. A little debt speeds development so long as it is paid back promptly with a rewrite … "

"The danger occurs when the debt is not repaid. Every minute spent on not-quite-right code counts as interest on that debt. Entire engineering organizations can be brought to a stand-still under the debt load of an unconsolidated implementation, object-oriented or otherwise. "

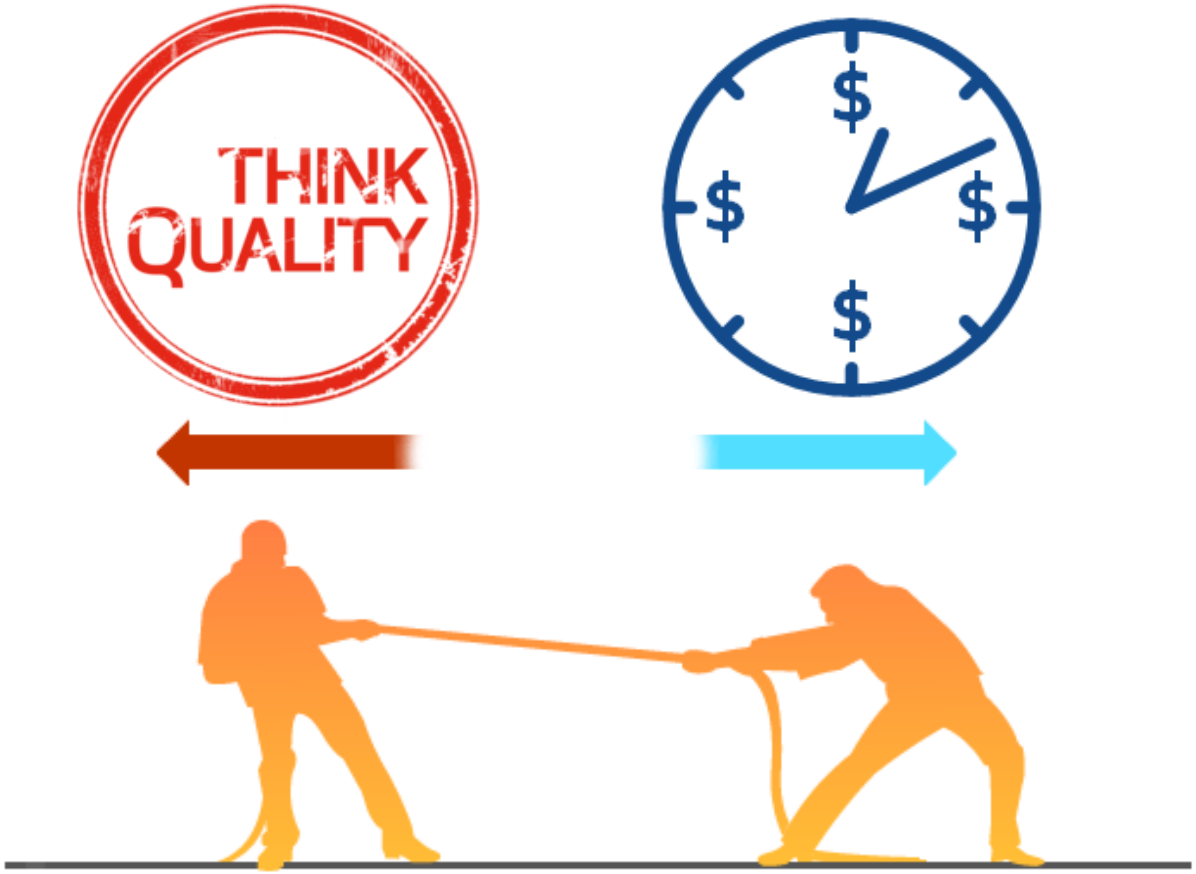
Ward Cunningham, The WyCash portfolio management system, OOPSLA '92

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

Technical Debt is a collection of design or implementation constructs
that are expedient in the short term,
but set up a technical context that
can make future changes more costly or impossible

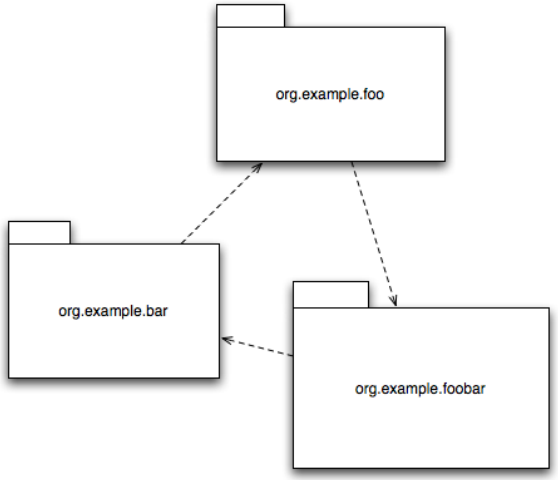
Dagstuhl April 2016

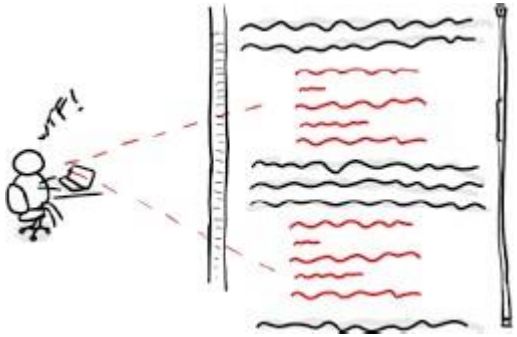
ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

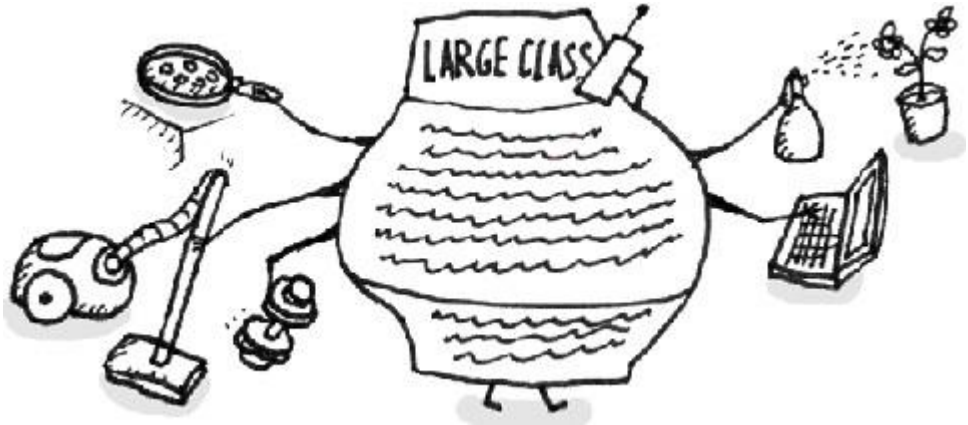


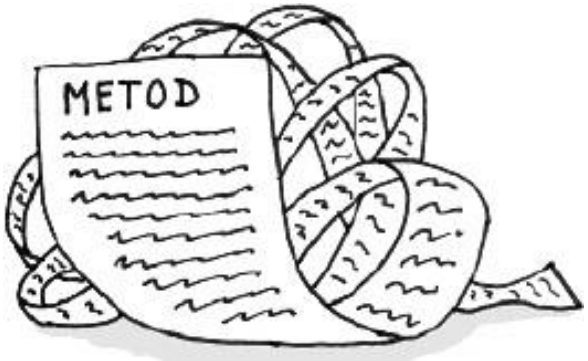
ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

Technical Debt illustrated









Images from https://refactoring.guru/smells

Technical Debt metaphor

- Debt is a necessary tradeoff
  - Loan for investment
  - Quality-- for business value++
- Pay back *principal* (fix TD) + *interest* (maintain SW)
- Debt should be monitored and managed
  - Risk – accumulation may spiral out of control

Typical symptoms

- Taking more time to build a feature or fix defects
- Changes ripple through the system
- Rework is often and unexpected
- Deadlines/milestones continuously slipping
- Velocity drops
- Testing becomes very expensive

Is this really new?

Communities

- Maintenance & evolution
- Reengineering / refactoring

Terms

- Aging
- Decay
- Sustainability

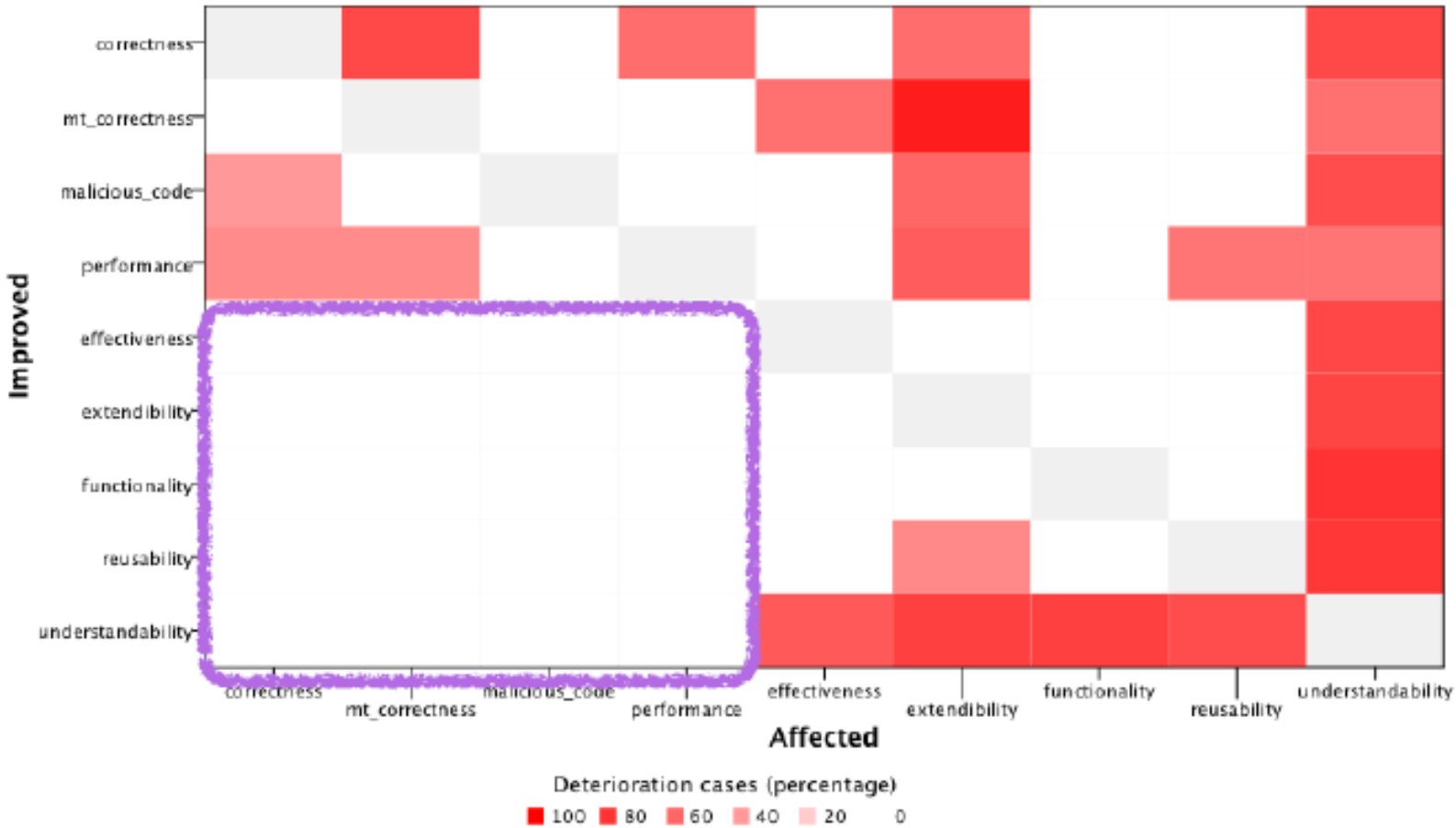
Is it important everywhere?

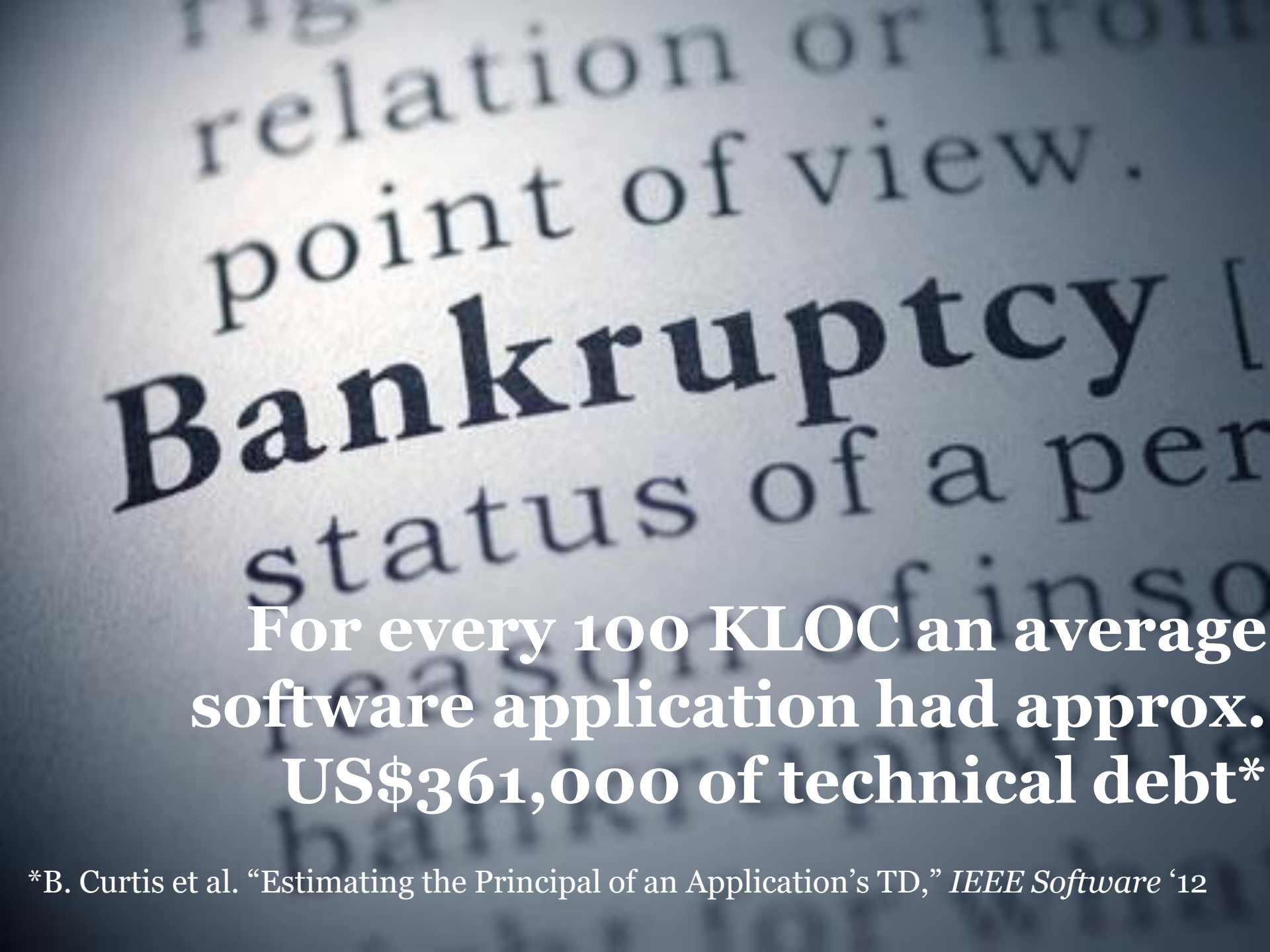
The technical debt management activities (TD repayment, prevention, etc.) are expected to be more relevant for projects for which long-term maintenance periods are anticipated.

- **Expected Lifetime:**
  <1 to 30 years
- **Mean Expected Lifetime:**
  12.40 years
- **Standard Deviation:**
  8.94
- **Median:**
  10 years

- **Short-term:** 45%
- **Long-term:** 55%

Estimated Lifespan		Maintainability				
		very low	Low	neutral	high	very high
Long	Observed Count	0,0	0,0	3,0	5,0	1,0
	Expected Count	1,4	1,4	1,4	4,5	0,5
Short	Observed Count	3,0	3,0	0,0	5,0	0,0
	Expected Count	1,7	1,7	1,7	5,5	0,6

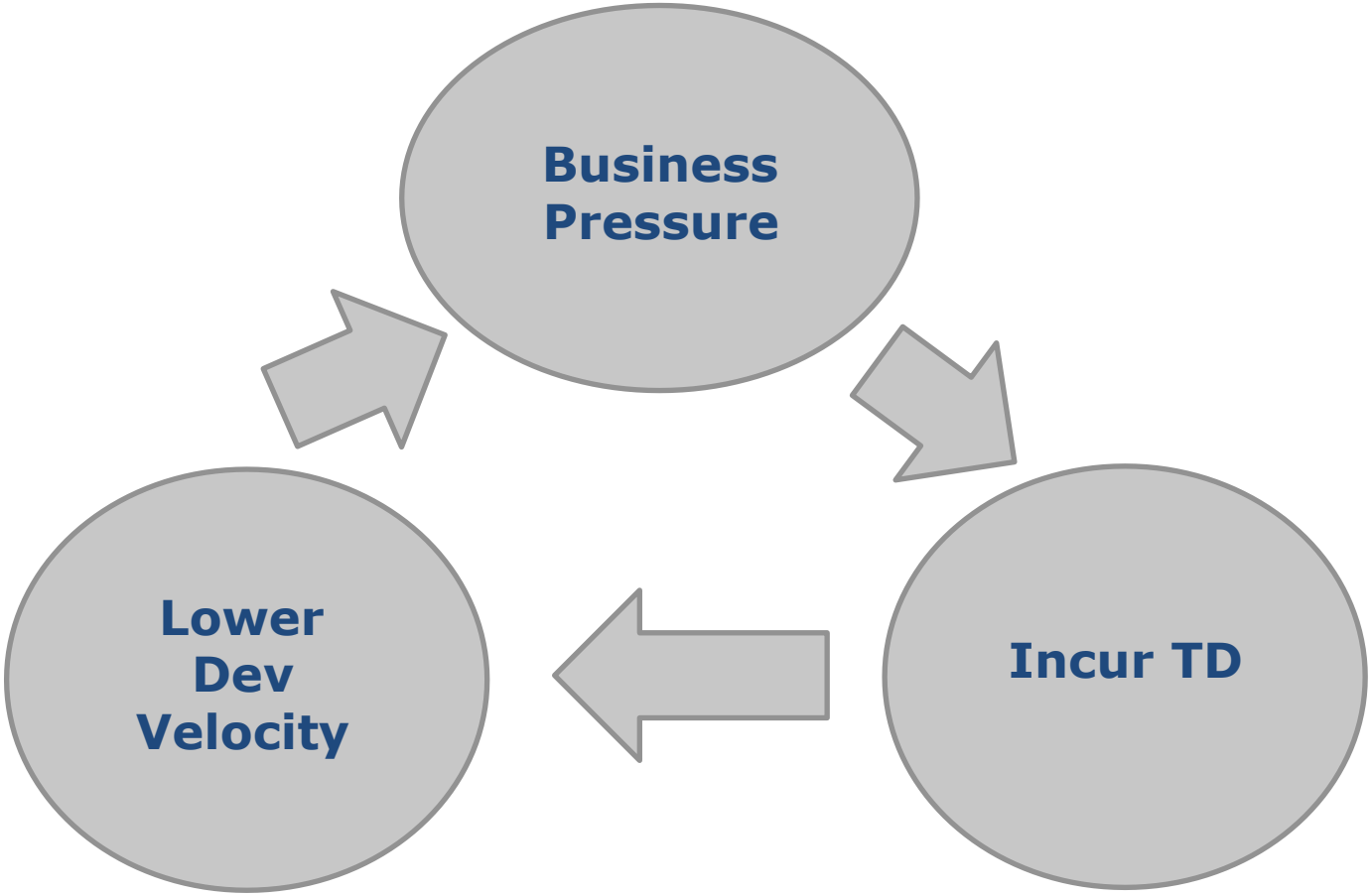

correctness
mt_correctness
malicious_code
performance
effectiveness
extendibility
functionality
reusability
understandability
Improved
Affected
Deterioration cases (percentage)
100
80
60
40
20
0



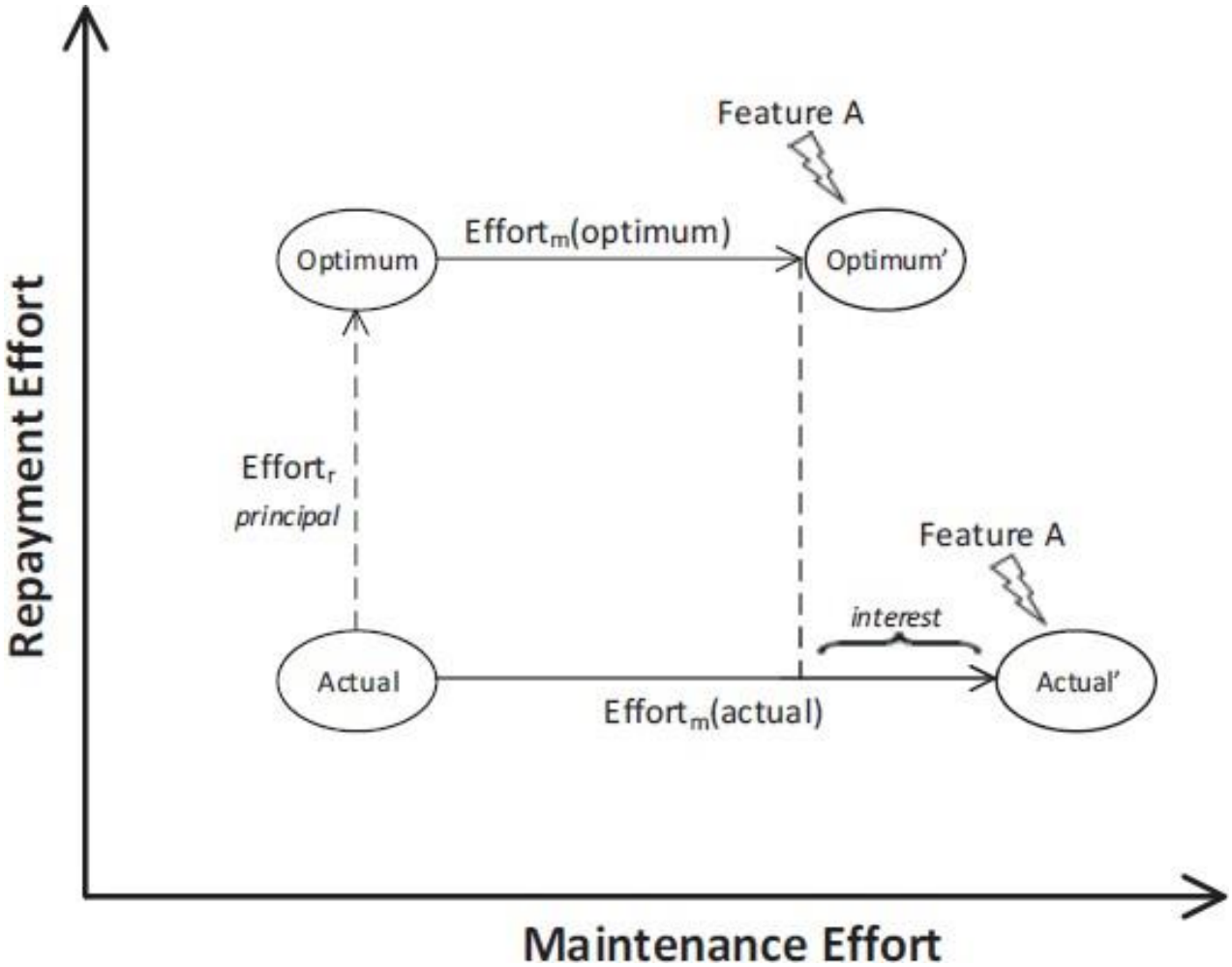
For every 100 KLOC an average software application had approx. US$361,000 of technical debt*

*B. Curtis et al. "Estimating the Principal of an Application's TD," *IEEE Software* '12

Vicious circle of technical debt

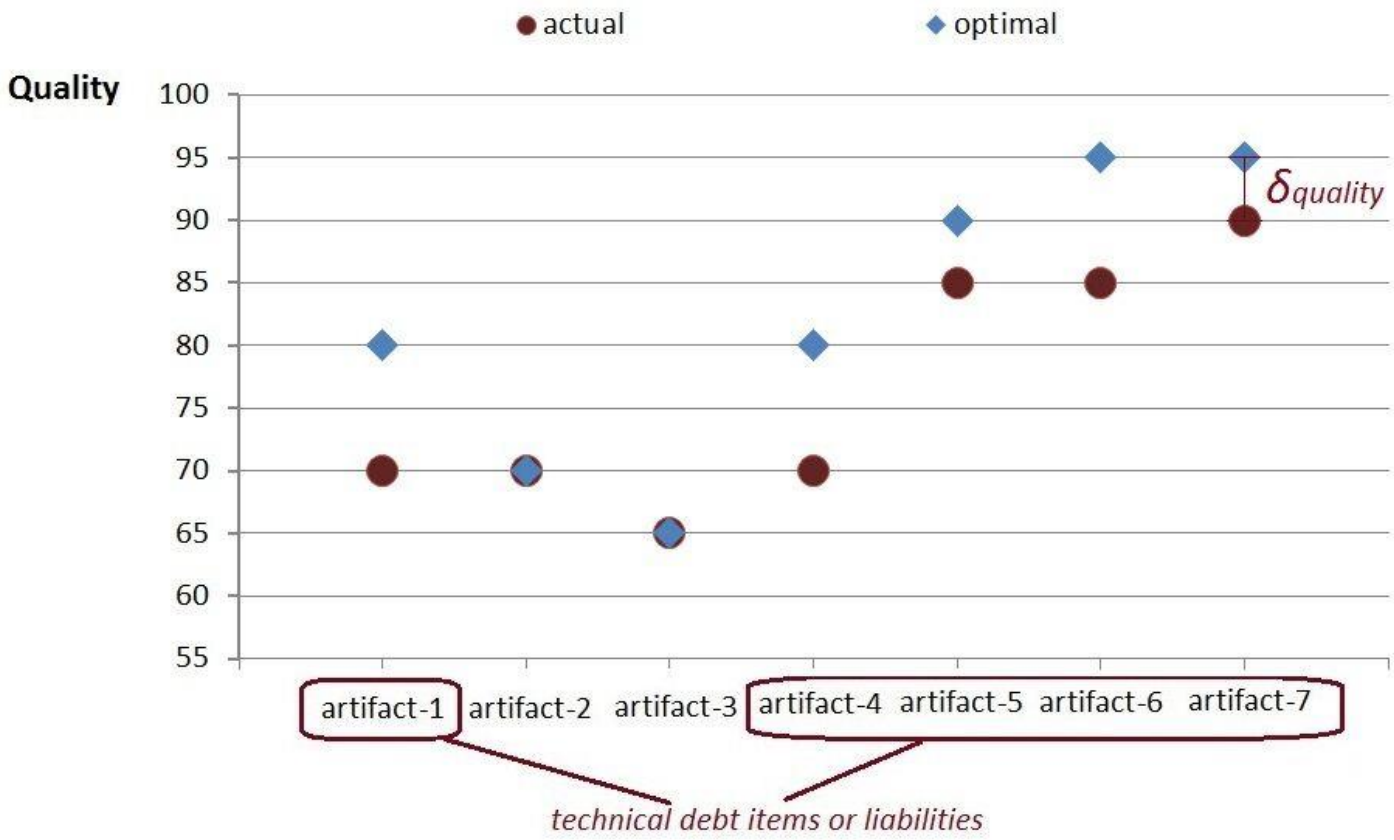


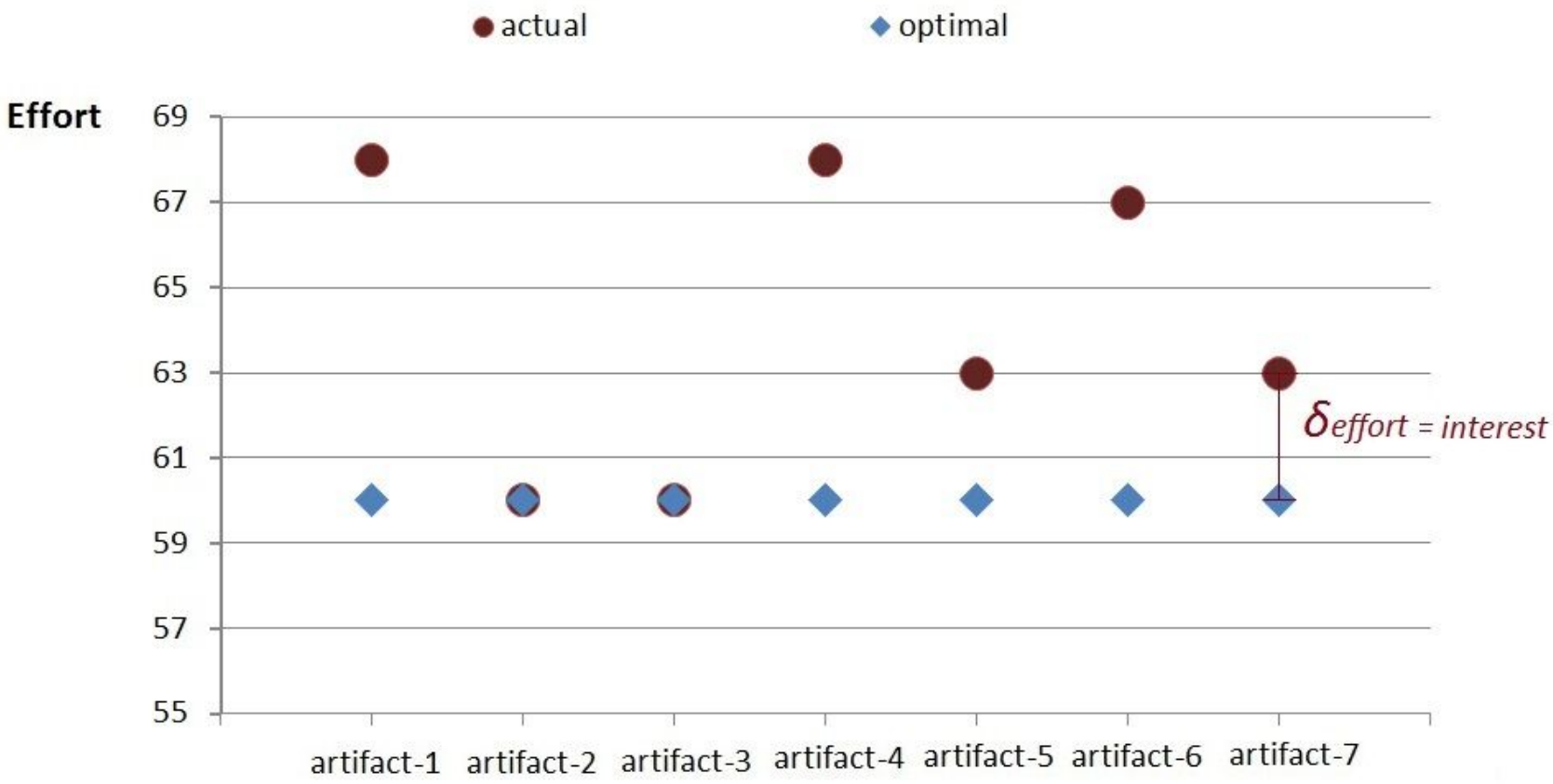
Debt=Principal+Interest

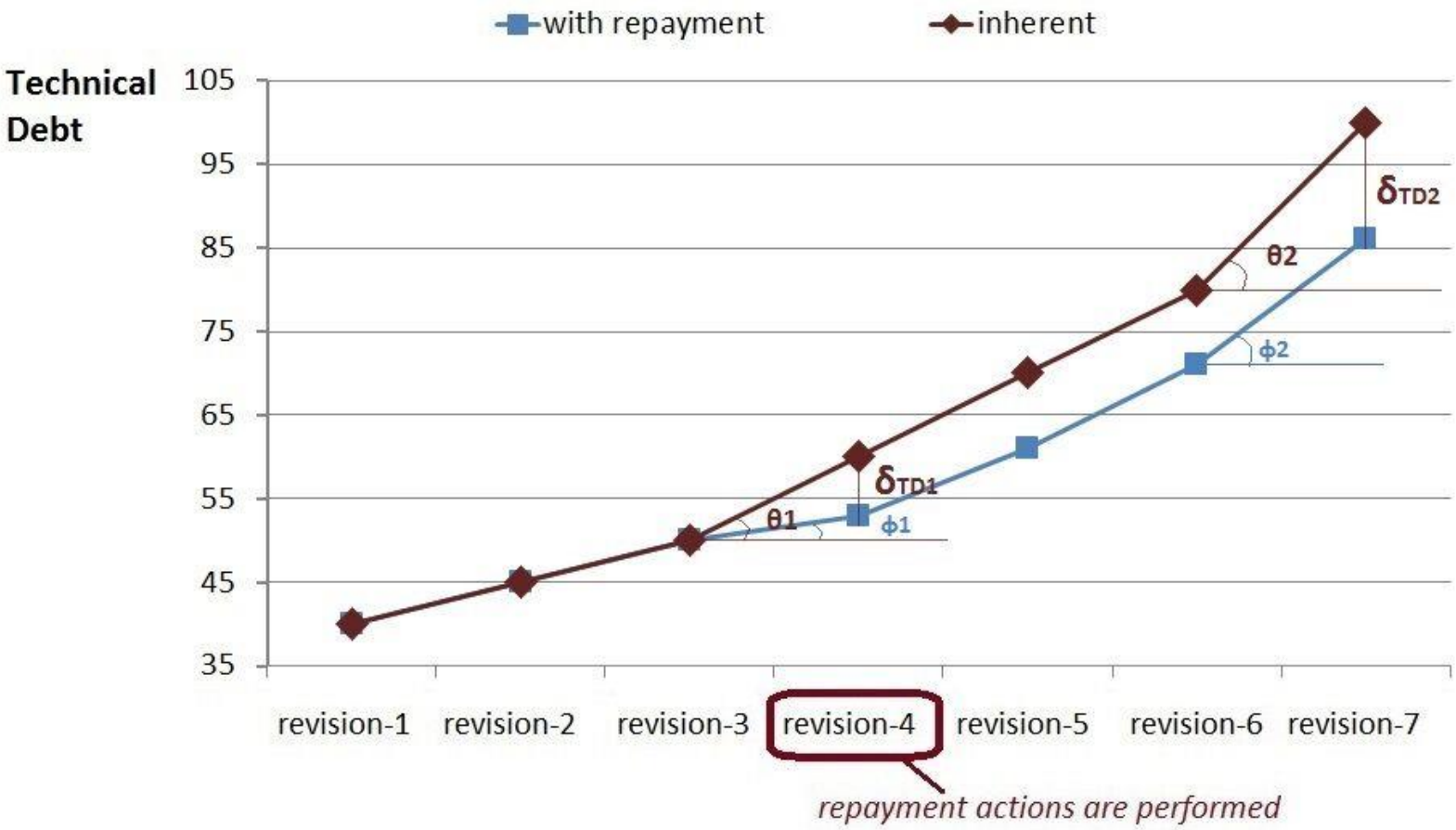


Ampatzoglou et al., A Financial Approach for Managing Interest in TD, BMSD '15

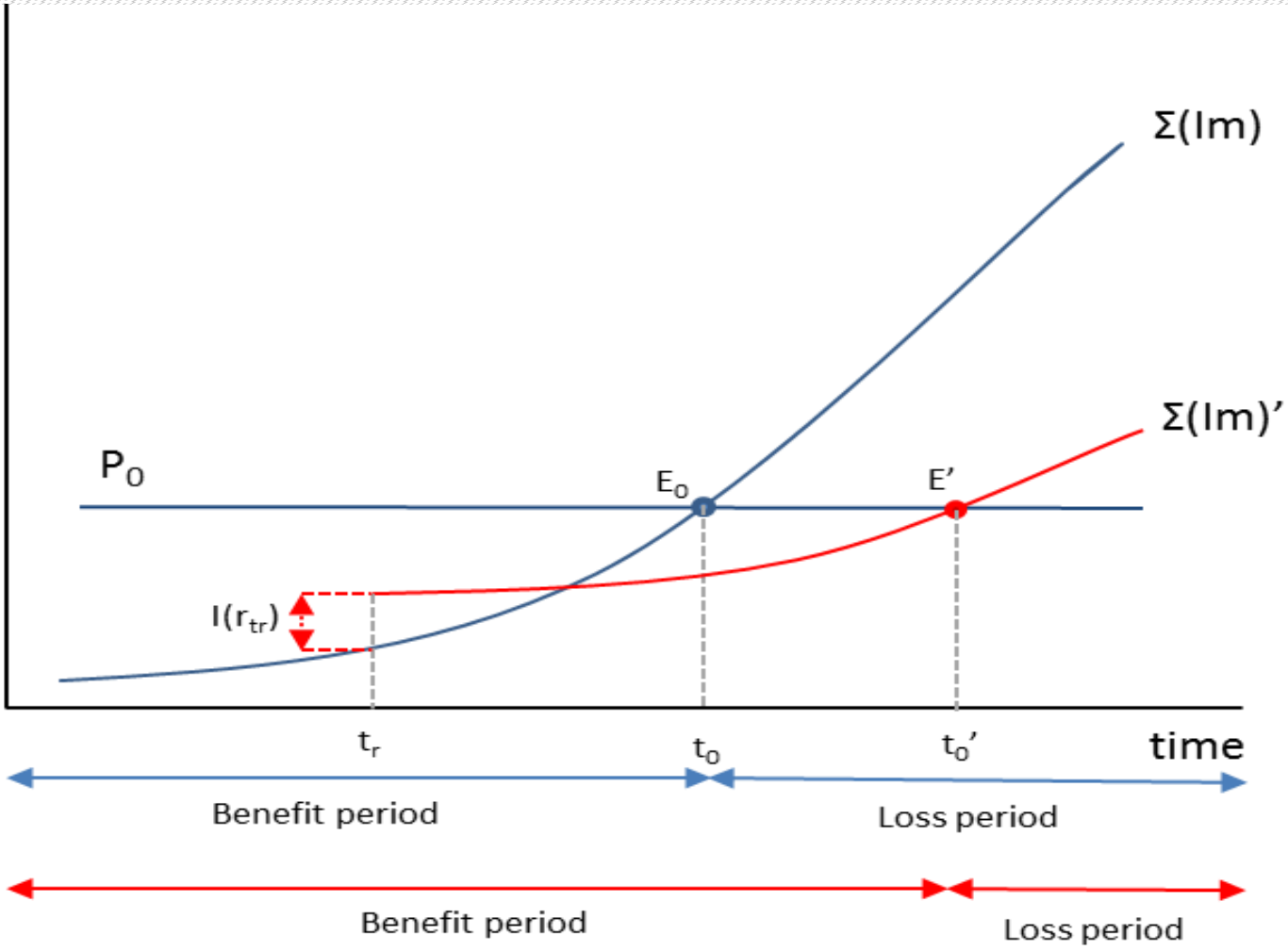
Basic Terms


actual
optimal
Quality
100
95
90
85
80
75
70
65
60
55
$\delta_{quality}$
artifact-1
artifact-2
artifact-3
artifact-4
artifact-5
artifact-6
artifact-7
technical debt items or liabilities


actual
optimal
Effort
69
67
65
63
61
59
57
55
artifact-1
artifact-2
artifact-3
artifact-4
artifact-5
artifact-6
artifact-7
δeffort = interest


with repayment
inherent
Technical Debt
105
95
85
75
65
55
45
35
δTD2
θ2
ϕ2
δTD1
θ1
ϕ1
revision-1
revision-2
revision-3
revision-4
revision-5
revision-6
revision-7
repayment actions are performed

Breaking point: principal vs. interest



Ampatzoglou et al., A Financial Approach for Managing Interest in TD, BMSD '15

Just Code?

Not quite right

- Code
- Requirements
- Architecture
- Design
- Test
- Build
- Documentation
- Infrastructure
- Versioning

…

Technical debt is pervasive

- Code
- Requirements
- Architecture
- Design
- Test
- Build
- Documentation
- Infrastructure
- Versioning

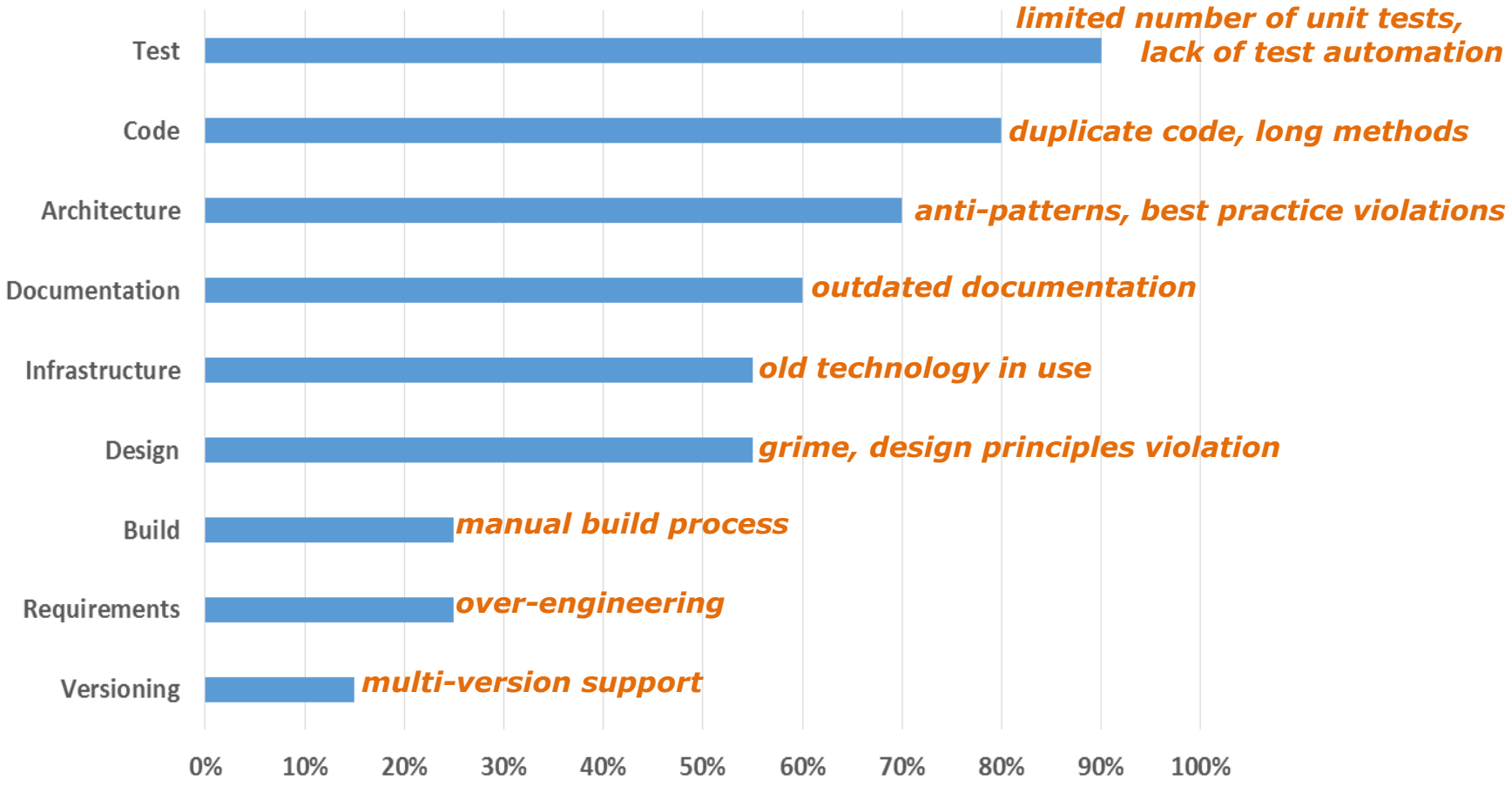
Complex dependencies
Architecture smells
Architecture drift

- Code
- Requirements
- Architecture
- Design
- Test
- Build
- Documentation
- Infrastructure
- Versioning

Low code coverage
Lack of test automation
Residual defects not found

- Code
- Requirements
- Architecture
- Design
- Test
- Build
- Documentation
- Infrastructure
- Versioning

Insufficient/incomplete/out of date
Lack of code comments


Test
limited number of unit tests, lack of test automation
Code
duplicate code, long methods
Architecture
anti-patterns, best practice violations
Documentation
outdated documentation
Infrastructure
old technology in use
Design
grime, design principles violation
Build
manual build process
Requirements
over-engineering
Versioning
multi-version support
0%
10%
20%
30%
40%
50%
60%
70%
80%
90%
100%

Debt

TDM Overview

- TD prevention aims to prevent potential TD from being incurred.
- TD identification detects TD caused by intentional or unintentional technical decisions in a software system through specific techniques, such as static code analysis.
- TD measurement quantifies the benefit and cost of known TD in a software system through estimation techniques, or estimates the level of the overall TD in a system.
- TD representation/documentation provides a way to represent and codify TD in a uniform manner.
- TD communication makes identified TD visible to stakeholders so that it can be discussed and managed.
- TD monitoring watches the changes of the cost and benefit of unresolved TD over time.
- TD prioritization ranks identified TD according to certain predefined rules to support deciding which TD items should be repaid first and which TD items can be tolerated until later releases.
- TD repayment mitigates TD in software, by techniques such as reengineering and refactoring.

Li et al., Architectural Debt Management in Value-oriented Architecting, Elsevier '14

Managing TD

- TD prevention
- TD identification
- TD measurement
- TD prioritization
- TD monitoring
- TD repayment
- TD representation/documentation
- TD communication

Code analysis
Dependency analysis
Solution comparison
Reverse engineering

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

TD Identification

sonarqube | Projects | Issues | Rules | Quality Profiles | Quality Gates | Administration | Search for projects, sub-projects and files...

My Favorites | All

Perspective: Overall Status | Sort by: Name | Search by project name or key | 15 projects

Filters

Quality Gate
- Passed 12
- Warning 0
- Failed 3

Reliability (Bugs)
- A 0
- B and worse 15
- C and worse 15
- D and worse 14
- E 14

Security (Vulnerabilities)
- A 1
- B and worse 14
- C and worse 10
- D and worse 10
- E 9

Maintainability (Code Smells)
- A 15
- B and worse 0
- C and worse 0
- D and worse 0
- E 0

Coverage
- ≥ 80% 0
- < 80% 15
- < 70% 15
- < 50% 15
- < 30% 15
- No data 0

Project	Status	Last analysis	Bugs	Vulnerabilities	Code Smells	Coverage	Duplications	Size	Language
beam	Passed	April 24, 2018, 5:39 AM	99 E	25 B	5.5k A	0.0%	1.4%	154k L	Java
bookkeeper	Passed	June 5, 2018, 9:53 AM	88 E	48 E	3.5k A	0.0%	2.0%	99k M	Java
commons-io	Failed	May 22, 2018, 7:05 PM	26 E	3 B	304 A	0.0%	3.2%	10k M	Java
commons-lang	Passed	May 22, 2018, 7:30 PM	25 C	5 B	723 A	0.0%	2.1%	26k M	Java
commons-math	Passed	May 16, 2018, 5:51 PM	76 E	0 A	1.9k A	0.0%	2.8%	83k M	Java
flink	Failed								

Applicability

TD Dashboard

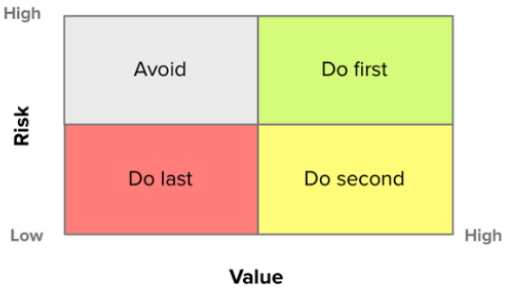
Visualizing TD

shows information that support:

artifacts that suffer from TD

measure their amount of interest and principal

Quantifying TD

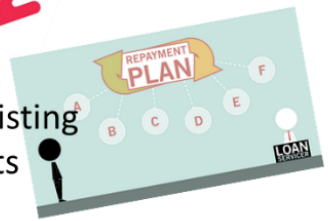


based on quantification

Prioritizing TD

prevent TD on new artifacts

refactor existing artifacts



Reducing TD

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

60

9

11

RQ1: *What could be the optimal information to be visualized in a TDM dashboard for efficient TD quantification?*

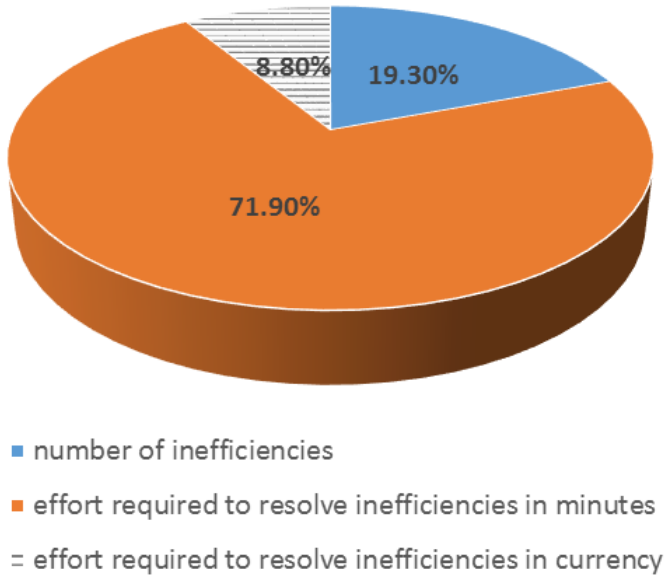
RQ2: *What could be the optimal information to be visualized in a TDM dashboard for efficient TD prioritization?*

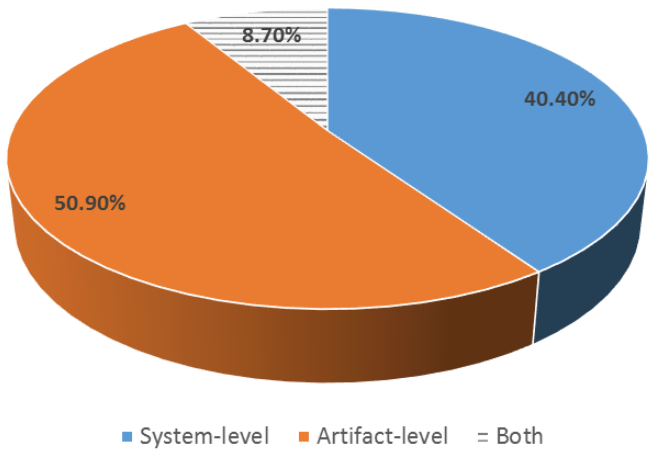
RQ3: *What could be the optimal information to be visualized in a TDM dashboard for efficient TD reduction?*

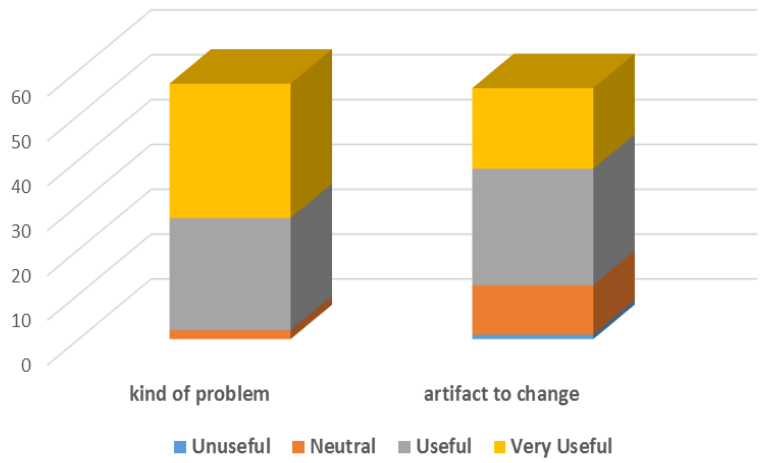
RQ4: *Do different roles of stakeholders require different views of the quality dashboard?*

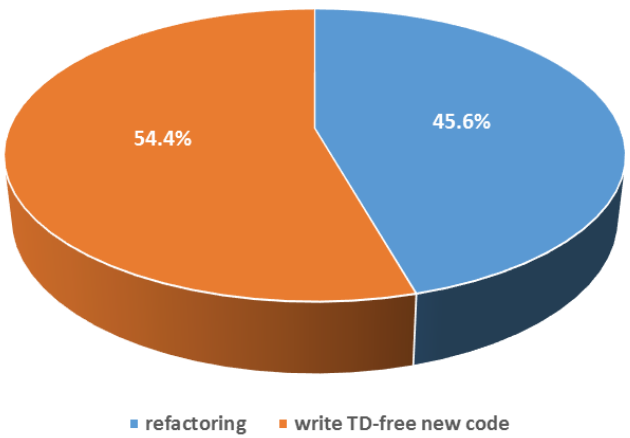
Survey Administration: Google Forms

ID	Question
Part 2- TD Principal	
Q2.1	How useful is it to know how many inefficiencies your code has?
Q2.2	How useful is it to have an estimation of the effort required to solve all inefficiencies?
Q2.3	How useful is it to have an estimation of the effort required to solve all inefficiencies in a currency format (e.g. $ or €)?
Q2.4	If you had only one option for quantifying inefficiencies, which one would you prefer?
Q2.5	To what extent is the granularity (architecture, method) of an inefficiency is important for selecting the ones to resolve?
Part 3- TD Interest/Interest Probability	
Q3.1	How useful is it to know how maintainable your code as a whole (system-level) is?
Q3.2	How useful is it to know the maintainability of specific artefacts?
Q3.3	How useful is it to know the kind of structural problems that hinder maintainability?
Q3.4	How useful is it to know the probability of one artefact to need maintenance?
Q3.5	At which level of granularity would you prefer to get a maintainability indicator?
Q3.6	What piece of information would be more important for selecting which artefact to refactor first?
Part 4- TD Reduction	
Q4.1	Do you consider TD repayment (refactoring) as a useful activity for improving quality?
Q4.2	Do you consider writing new code that is TD-free as a useful activity for improving quality?
Q4.3	Do you consider a metric comparing TD density on existing and new code useful?
Q4.4	To reduce the TD density of your system, would you prefer to refactor or write TD-free new code?


8.80%
19.30%
71.90%
number of inefficiencies
effort required to resolve inefficiencies in minutes
effort required to resolve inefficiencies in currency


8.70%
40.40%
50.90%
System-level
Artifact-level
Both


60
50
40
30
20
10
0
kind of problem
artifact to change
Unuseful
Neutral
Useful
Very Useful


54.4%
45.6%
refactoring
write TD-free new code

Managing TD

- TD prevention
- TD identification
- TD measurement
- TD prioritization
- TD monitoring
- TD repayment
- TD representation/documentation
- TD communication

Mathematical models
Code metrics
Human estimation

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

TD Principal

sonarqube | Projects | Issues | Rules | Quality Profiles | Quality Gates | Administration | Search for projects, su

commons-io | master | May 22, 2018, 7:05 PM | Version 7791

Overview | Issues | Measures | Code | Activity | Administration

Quality Gate: Failed

0.0% Coverage on New Code is less than 80.0%

Bugs | Vulnerabilities

Leak Period: since bc10af423f2eaef8cfffe1c3ff06956afa3dd371 started last month

Bugs	Vulnerabilities	New Bugs	New Vulnerabilities
26 E	3 B	0 A	0 A

Code Smells

Debt	Code Smells	New Debt	New Code Smells
7d A	304	0 A	0

started 12 years ago

Coverage

Coverage	Coverage on 26 New Lines to Cover
0.0%	0.0%

Duplications

Duplications	Duplicated Blocks	Duplications on 385 New Lines
3.2%	22	0.0%

The Apa
utility cla
filters, fi
transfor

M

Lines

No ta

Activity

May 22,
7791a
Quality G

May 18,
bc10a
Quality G

April 29,
a4705
Quality G
Show Mo

Quality
(Default)

Quality
(Java) S

Hom
Cont
Bug
Sour
Deve

TD Interest

Interest ~ Maintainability

ISO 25010: Modifiability. Degree to which a product or system can be effectively and efficiently modified without introducing defects or degrading existing product quality

Metric	Description	Quality Attribute
DIT	Depth of Inheritamnce Tree: Inheritance level number, 0 for the root class	*inheritance*
NOCC	Number of Children Classes: Number of direct sub-classes that the class has.	*inheritance*
MPC	Message Passing Coupling: Number of send statements defined in the class	*coupling*
RFC	Response For a Class: Number of local methods plus the number of methods called by class methods	*coupling*
LCOM	Lack of Cohesion of Methods: Number of disjoint sets of methods in the class	*cohesion*
DAC	Data Abstraction Coupling: Number of abstract types defined in the class	*coupling*
WMPC	Weighted Method per Class: Average cyclomatic complexity of all methods in the class	*complexity*
NOM	Number of Methods: Number of methods in the class	*size*
SIZE1	Lines of Code: Number of semicolons in the class	*size*
SIZE2	Number of Properties: Number of attributes and methods in the class	*size*

Managing TD

- TD prevention
- TD identification
- TD measurement
- TD prioritization
- TD monitoring
- TD repayment
- TD representation/documentation
- TD communication

Refactoring
Automating manual tasks

Refactoring

sonarqube | Projects | Issues | Rules | Quality Profiles | Quality Gates | Administration | Search for projects, sub-projects and files...

commons-io | master | June 17, 2018, 12:51 AM | Version 078af456fd74c3726438bfe853492286d7006c06

Overview | Issues | Measures | Code | Activity | Administration

My Issues | All | Bulk Change | ↑ ↓ to select issues | ← → to navigate | 1 / 95 issues

Filters | Clear All Filters

Display Mode: Issues | Effort

Type
- Bug 8
- Vulnerability 0
- Code Smell 87

Severity | Clear
- Blocker 6
- Critical 67
- Major 95
- Minor 94
- Info 74

Resolution

Status

src/.../java/org/apache/commons/io/Charsets.java
- Add a private constructor to hide the implicit public one. — 6 years ago L56 — Code Smell Major Open Not assigned 30min effort Comment — design

src/.../java/org/apache/commons/io/CopyUtils.java
- Hide this public constructor. — 15 years ago L122 — Code Smell Major Open Not assigned 30min effort Comment — design

src/.../java/org/apache/commons/io/EndianUtils.java
- Hide this public constructor. — 13 years ago L45 — Code Smell Major Open Not assigned 30min effort Comment — design

src/.../java/org/apache/commons/io/FileCleaner.java
- Add a private constructor to hide the implicit public one. — 14 years ago L38 — Code Smell Major Open Not assigned 30min effort Comment — design

src/.../java/org/apache/commons/io/FileCleaningTracker.java

Utility classes should not have public constructors

Code Smell | Major | design | Available Since June 29, 2018 | Constant/issue: 30min

Utility classes, which are collections of `static` members, are not meant to be instantiated. Even abstract utility classes, which can be extended, should not have public constructors.

Java adds an implicit public constructor to every class which does not define at least one explicitly. Hence, at least one non-public constructor should be defined.

Noncompliant Code Example

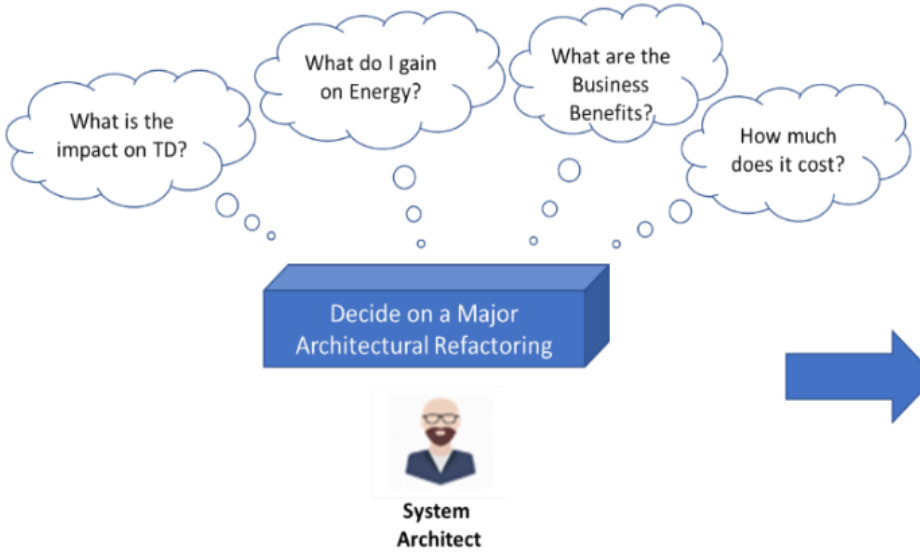
```
class StringUtils { // Noncompliant

  public static String concatenate(String s1, String s2) {
    return s1 + s2;
  }

}
```

Compliant Solution

- Large variation in survivability of issues
  - 10% fixed within the first month
  - 50% in the first year
  - Some take up to ten years
- Very few issues types with fixing rate >50%
- Duplication and exception handling
  - Frequently encountered
  - Rarely fixed

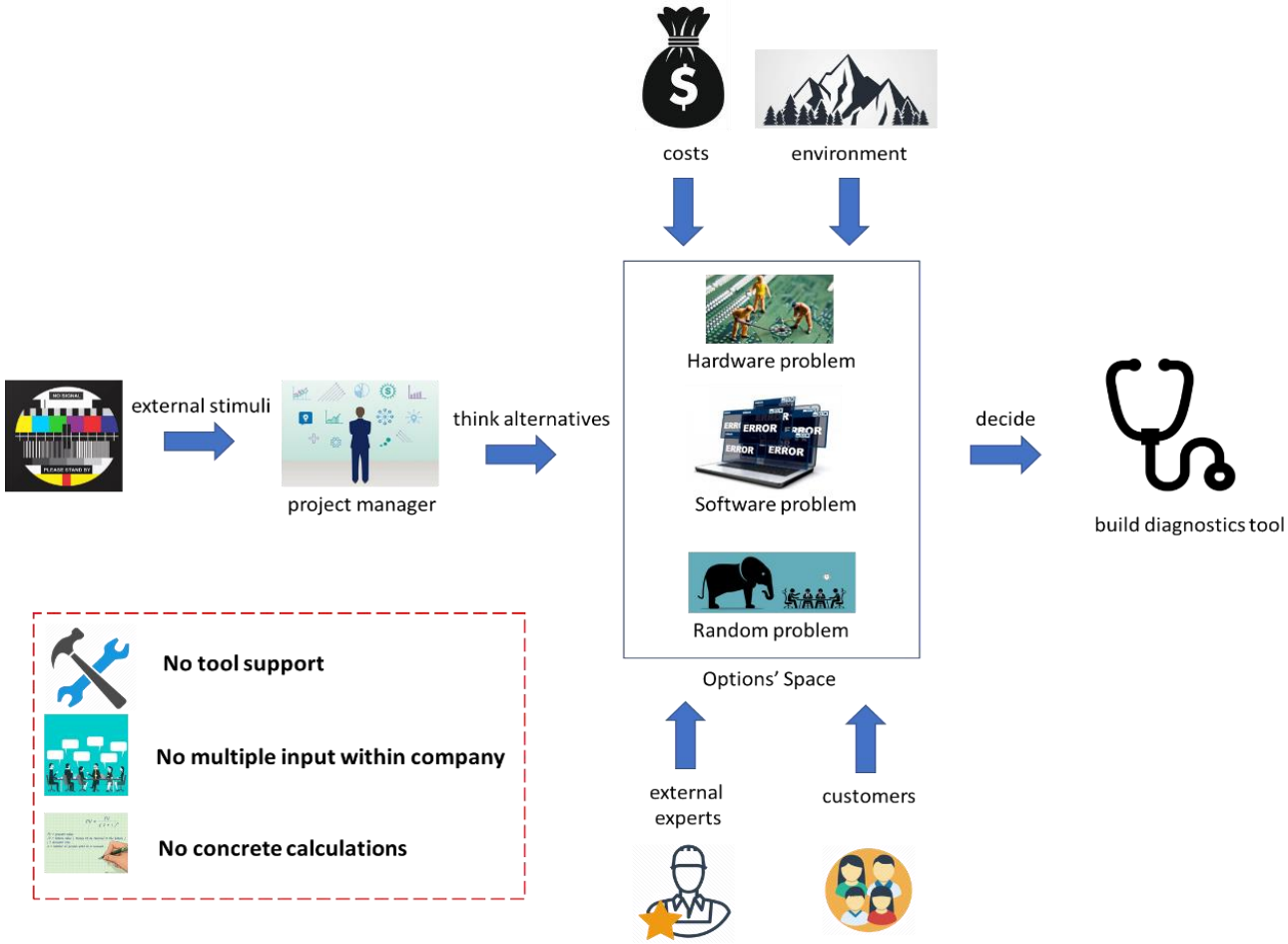
Digkas et al., How Do Developers Fix Issues and Pay Back TD in the Apache Ecosystem, SANER '18

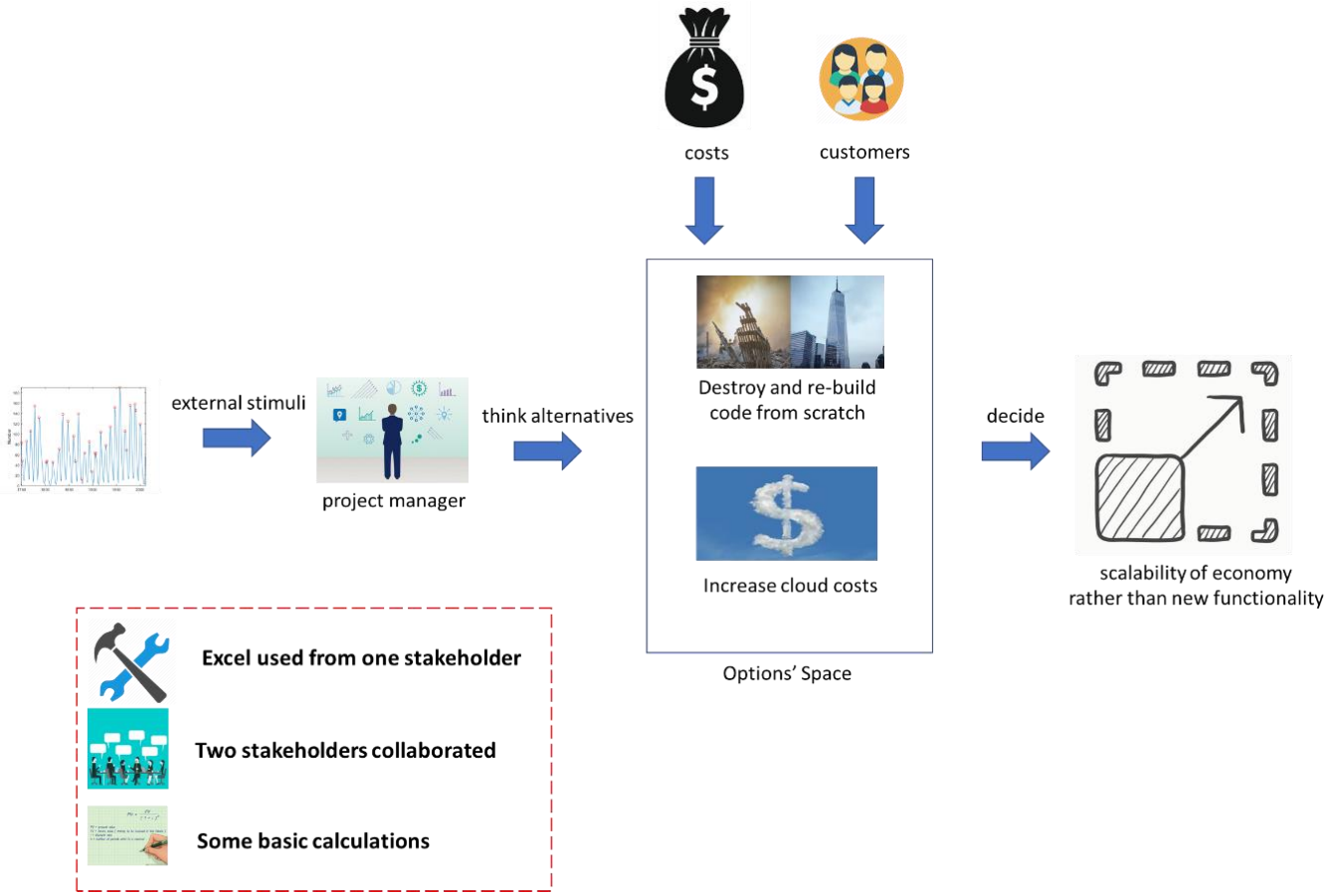
Decision-Making in Industry



Cost-Benefit Analysis Model

- Cost to Apply Refactoring (P)
- Cost to Measure Phenomena (A)
  - TD
  - Energy
- Cost in Terms of Extra Interest (F)
- Benefits due to Market Share (B)
- Benefits due to Increased Price (B)
- Benefits due to Entrance in Market (B)
- Benefits due to Customer Relations (B)


costs
environment
external stimuli
project manager
think alternatives
Hardware problem
Software problem
Random problem
Options' Space
decide
build diagnostics tool
external experts
customers
No tool support
No multiple input within company
No concrete calculations


costs
customers
external stimuli
project manager
think alternatives
Destroy and re-build code from scratch
Increase cloud costs
Options' Space
decide
scalability of economy rather than new functionality
Excel used from one stakeholder
Two stakeholders collaborated
Some basic calculations

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

Not allow new TD

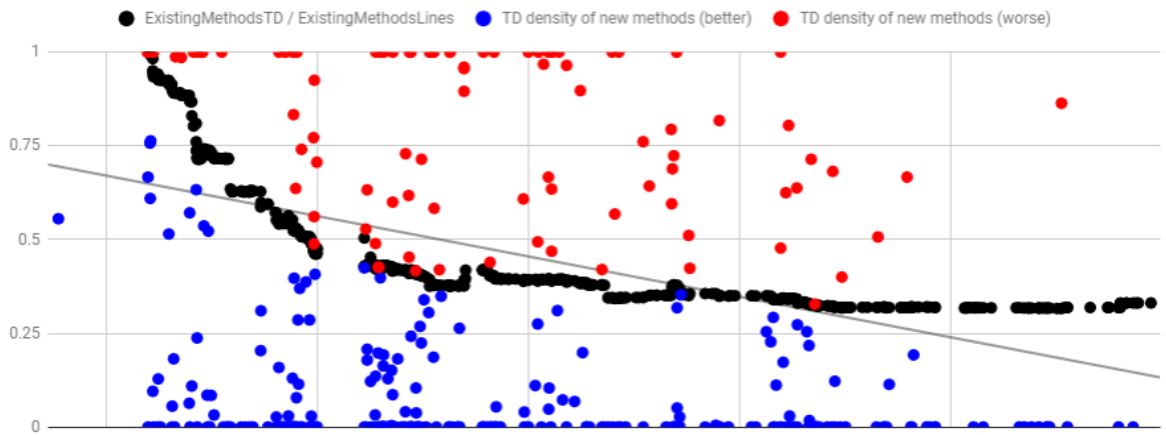
- TD prevention
- TD identification
- TD measurement
- TD prioritization
- TD monitoring
- TD repayment
- TD representation/documentation
- TD communication

New Code TD: Motivation

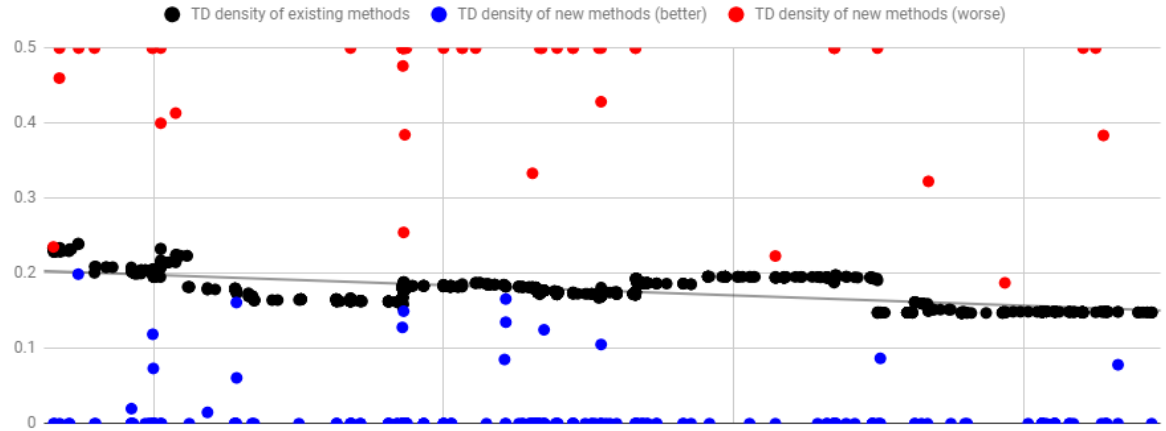
Evolution in Absolute Values

Evolution in Normalized Values

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ



commons-io

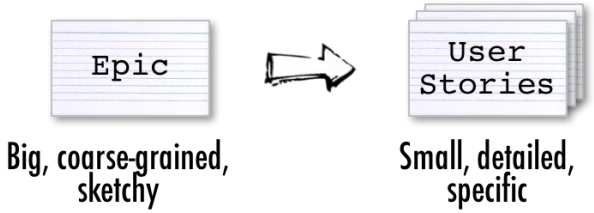


giraph

Documentation TD in Requirements

› Stakeholders

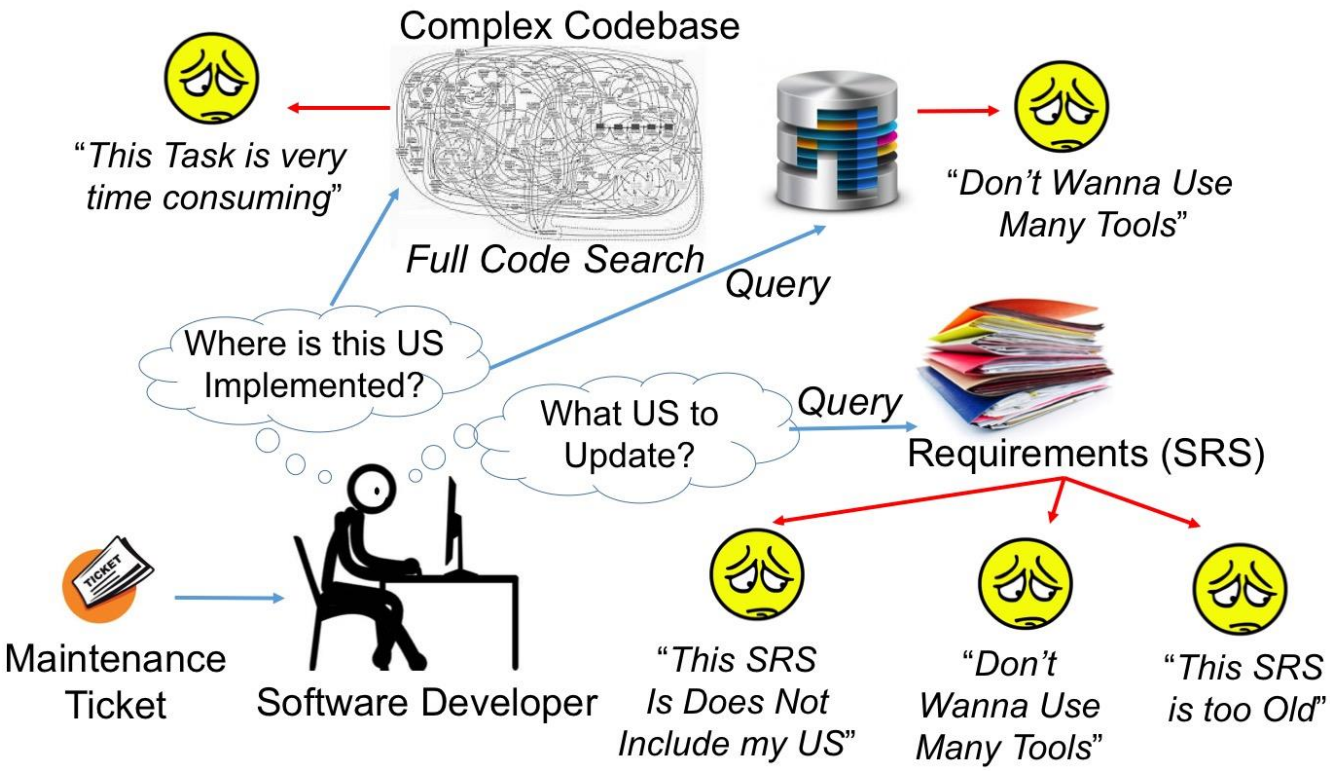
› Methods & Tools



Jira

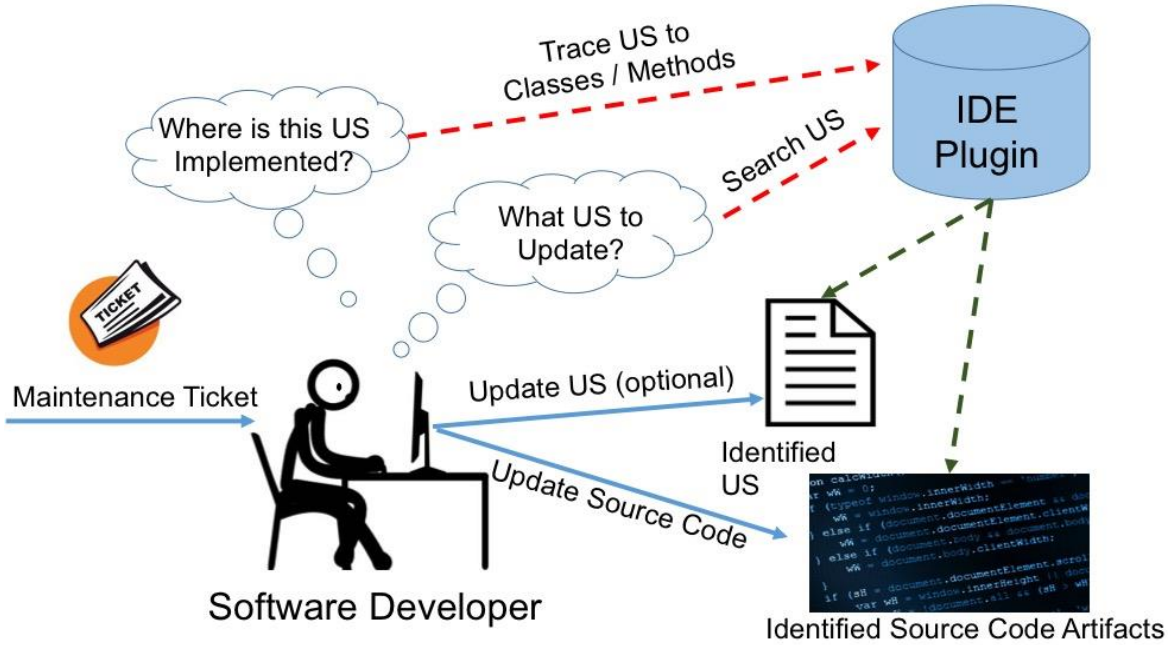
Why to Accumulate TD

Insufficient Requirements – Incomplete Requirements – Outdated Requirements

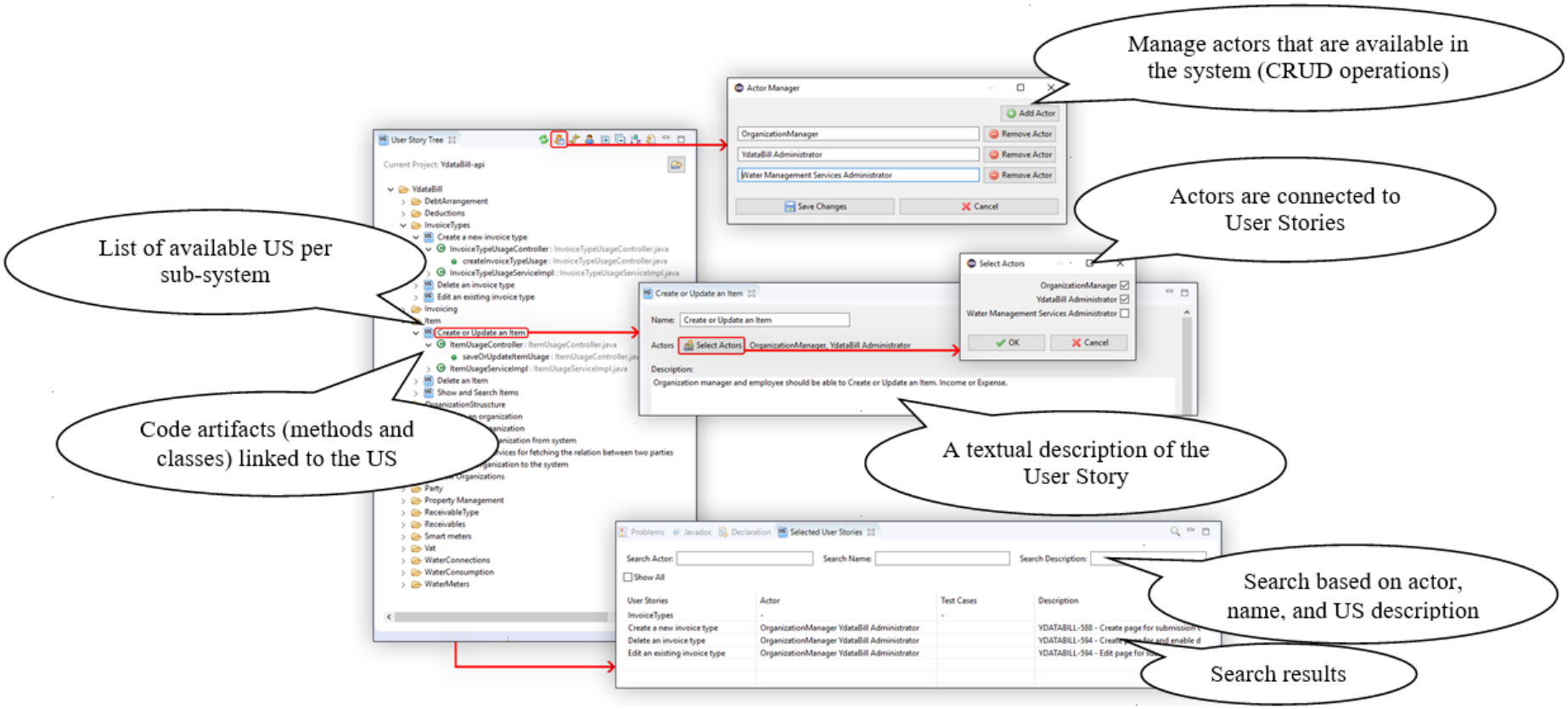


Prevention through Tracing

Main Usage Scenario



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

Ερωτήσεις